

Name : Aryan Ashraf

Roll No. : 22609

Class : BBA

Semester : (V)

Assignment (1) : Web Engineering

Institute : Government Graduate College
of Commerce SKP.

What is World Wide Web?

World Wide Web, which is also known as a Web, is a collection of websites or web pages stored in web servers and connected to local computers through the internet. These websites contain text pages, digital images, audio, videos, etc. Users can access the content of these sites from any part of the world over the internet using their devices such as computers, laptops, cell phones, etc. The WWW, along with the internet, enables the retrieval and display of text and media to your device.

Key Features:

1. Links between web pages are created using hyperlinks.
2. Users can access multimedia content like text, images, videos, and more.
3. It operates on the client-server model, where browsers act as clients and web servers provide resources.

History:

- Invented by Tim Berners-Lee in 1989.
- First website was launched in 1991, hosted on CERN servers.

Internet Protocols

Internet protocols are the set of rules or standards that govern how data is transmitted, received, and processed across the internet. These protocols ensure that devices, such as computers, smartphones, and servers, can communicate with each other effectively, regardless of their hardware or operating systems.

Examples of internet protocols include TCP/IP, HTTP, FTP, SMTP, POP, IMAP, and DNS, each serving specific purposes like web browsing, email transmission, or file sharing. Without internet protocols, devices would not be able to understand or interact with each other.

1. TCP/IP (Transmission Control Protocol/Internet Protocol):

TCP/IP is the backbone of the internet. It is a set of rules that control how data is broken into small packets, sent, and reassembled at the destination. TCP ensures the data is sent accurately, while IP handles the addresses to make sure the data reaches the correct destination. It is used for most internet communications, including web browsing and file sharing.

2. SMTP (Simple Mail Transfer Protocol):

SMTP is the protocol used to send emails. When you write and send an email, SMTP helps transfer the message from your computer or email app to the email server. It also handles the delivery of the email to the recipient's email server. However, it doesn't handle receiving emails.

3. UDP (User Datagram Protocol):

UDP is a protocol for sending data over the internet. Unlike TCP, it doesn't check if all data packets arrive or are in the correct order. This makes it faster but less reliable. It is commonly used in applications where speed matters, like online gaming, video calls, and live streaming.

4. PPP (Point-to-Point Protocol):

PPP is a protocol that establishes a direct connection between two computers over a phone line, cable, or wireless connection. It's mostly used for dial-up internet connections and is also useful for connecting a computer to the internet or a private network.

5. POP (Post Office Protocol):

POP is a protocol used by email clients to download emails from a mail server. When you check your emails, POP downloads them to your computer or phone and usually deletes them from the server afterward. This means the emails are stored locally and may not be accessible on other devices.

6. IMAP (Internet Message Access Protocol):

IMAP is another protocol for receiving emails, but it works differently from POP. Instead of downloading and deleting emails, IMAP keeps the messages on the server. This allows you to access the same emails from multiple devices, such as your phone, tablet, and computer, and keep them in sync.

7. HTTP (HyperText Transfer Protocol):

HTTP is the protocol used for loading web pages on the internet. When you type a website's address into your browser, HTTP helps your browser request the web page from the server and display it. It's used for most websites but doesn't encrypt the data being sent or received.

8. FTP (File Transfer Protocol):

FTP is used to transfer files between computers over the internet or a network. It allows you to upload files to a website's server or download files from it. FTP is commonly used by website administrators to manage files on their servers.

9. DNS (Domain Name System):

DNS acts as the internet's directory. It translates human-readable website names (like `www.example.com`) into IP addresses (like `192.168.1.1`) that computers use to find and connect to the correct servers. Without DNS, you would have to memorize long IP addresses to visit websites.

Internet Services

Internet services are a variety of tools and functionalities provided over the internet, which have become essential for modern life. These services

facilitate communication, entertainment, learning, and business activities. Examples of internet services include:

- **Email:** A fast and convenient way to send and receive messages through platforms like Gmail, Yahoo, and Outlook. It is widely used for personal and professional communication.
- **Web Browsing:** Accessing and exploring websites to gather information, read news, or view multimedia content. Search engines like Google or Bing enhance this experience.
- **Social Media:** Platforms like Facebook, Instagram, and Twitter connect people worldwide, enabling them to share updates, photos, and videos. Social media is also used for marketing and networking.
- **Streaming Services:** Watching videos on YouTube or Netflix and listening to music on Spotify or Apple Music. These platforms provide on-demand entertainment.
- **Cloud Services:** Storing and accessing files online through platforms like Google Drive, Dropbox, or iCloud. These services ensure data is safe and accessible from anywhere.
- **Online Shopping:** Websites like Amazon, eBay, or Daraz make it easy to purchase goods and services from the comfort of home, offering a variety of options and convenient payment methods.
- **Gaming:** Online games enable players to interact with others in real-time, either for fun or competitive purposes. Popular platforms include Steam and PlayStation Network.

Web Browsers

Web browsers are software applications designed to enable users to access and interact with content on the World Wide Web. They serve as a bridge between users and online information by interpreting web languages like HTML, CSS, and JavaScript to present websites in a readable and interactive format.

Features:

- **Navigation:** Browsers allow users to visit different websites by typing URLs or using bookmarks.
- **Rendering:** They display web pages by interpreting code written in web languages.

- **Security:** Modern browsers offer secure connections (HTTPS) and block unsafe content.

Example:

Google Chrome: Known for its speed, simplicity, and extensive extension library.

Mozilla Firefox: Focuses on privacy and customization options for users.

Microsoft Edge: A modern browser integrated with Windows for seamless performance.

Safari: Apple's browser optimized for macOS and iOS devices.

Markup Languages and Scripting Languages

Markup Languages:

Markup languages are used to organize and structure the content of web pages. They define the layout and formatting of text, images, and other elements using tags. These tags help browsers understand how to display the content properly.

- **Examples:**

- **HTML (HyperText Markup Language):** Used to create the basic structure of web pages.
- **XML (eXtensible Markup Language):** Helps store and transport data in a structured way, often used for data sharing between systems.

Scripting Languages:

Scripting languages add interactivity and advanced functionality to websites. They allow the creation of dynamic web pages by enabling real-time updates, animations, form validation, and more. These scripts can run in the browser (client-side) or on a server (server-side).

- **Examples:**

- **JavaScript:** Runs in the browser to handle interactive features like drop-down menus or sliders.
- **PHP:** Executes on the server to manage back-end tasks, like interacting with databases or processing forms.

Difference:

Aspect	Markup Languages	Scripting Languages
Purpose	Used to define and structure the content of web pages.	Used to add interactivity and functionality to web pages.
Functionality	Focuses on organizing and presenting data.	Focuses on dynamic behavior and automation of tasks.
Execution	Interpreted directly by web browsers.	Requires a browser or server to execute the scripts.
Syntax	Uses tags enclosed in angle brackets (e.g., <tag>).	Uses programming-like syntax with commands and logic.
Examples	HTML, XML	JavaScript, PHP, Python
Use Cases	Structuring text, images, links, tables, and forms.	Validating forms, creating animations, and handling databases.
Dynamic Behavior	Cannot create dynamic or interactive features.	Enables dynamic updates and real-time changes on web pages.
Integration	Works with scripting languages to display structured data.	Works with markup languages to enhance functionality.

Complexity	Relatively simple and easy to learn.	Requires knowledge of logic, variables, and programming.
Output	Defines what the web page looks like.	Defines how the web page behaves or interacts with users.

HTML (HyperText Markup Language)

HTML Stands for hypertext markup language. It is a widely used type of markup language. HTML is the foundational language used to create and design web pages. It defines the structure and layout of a website using a series of tags. These tags tell web browsers how to display the content, ensuring that the web page appears correctly for users.

Features:

- **Tag-Based Structure:** HTML uses tags enclosed in angle brackets (e.g., <h1>, <p>,) to define elements like headings, text, and images.
- **Headings and Paragraphs:** Organize content into sections and make it easier to read.
- **Links and Navigation:** Enable users to move between web pages using clickable links.
- **Multimedia Support:** Allows embedding of images, audio, and videos to make websites engaging.

Structure of HTML:

- <!DOCTYPE>
- <html>
- <head>
- <title>Web page title</title>
- </head>
- <body>
- <h1>Write Your First Heading</h1>
- <p>Write Your First Paragraph.</p>
- </body>

- `</html>`

Description of HTML Example

<!DOCTYPE>: It defines the document type or it instructs the browser about the version of HTML.

<html >: This tag informs the browser that it is an HTML document. Text between html tag describes the web document. It is a container for all other elements of HTML except <!DOCTYPE>

<head>: It should be the first element inside the <html> element, which contains the metadata (information about the document). It must be closed before the body tag opens.

<title>: As its name suggested, it is used to add title of that HTML page which appears at the top of the browser window. It must be placed inside the head tag and should close immediately. (Optional)

<body> : Text between body tag describes the body content of the page that is visible to the end user. This tag contains the main content of the HTML document.

<h1> : Text between <h1> tag describes the first level heading of the webpage.

<p> : Text between <p> tag describes the paragraph of the webpage.

Example:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>My First Web Page</title>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome to My Website</h1>
```



```
<p>This is a paragraph of text on my web page.</p>



</body>

</html>
```

Tags In HTML

Heading Tag: Used for headings in a webpage. Ranges from <h1> (largest) to <h6> (smallest).

Example:

```
<h1>Main Heading</h1>

<h2>Subheading</h2>
```

Paragraph Tag: Used for writing paragraphs.

Example:

```
<p>This is a paragraph.</p>
```

Line Break and Paragraph Break

-
: Inserts a line break.
- <p>: Ends the current paragraph and starts a new one.

Example:

```
<p>This is the first paragraph.</p>

<p>This is the second paragraph.</p>
```

Ordered and Unordered List Tags

- **Ordered List** (): Displays items in a numbered list.
- **Unordered List** (): Displays items in a bulleted list.

Example:

- **Ordered List**

```
<ol>

  <li>Item 1</li>

  <li>Item 2</li>

</ol>
```

- **Unordered List**

```
<ul>

  <li>Point A</li>

  <li>Point B</li>

</ul>
```

HTML Table

HTML tables are used to organize and display data in a structured, tabular format. Tables are made up of rows and columns, with specific tags to define each part of the table.

Key Table Tags and Their Functions

Tag	Description
<table>	Defines the start and end of the table.
<tr>	Defines a table row. It contains table data (<td>) or header (<th>) cells.
<th>	Defines a header cell. Text in header cells is bold and centered by default.
<td>	Defines a table data cell. It holds the content of the table (text, images, or other elements).
<caption>	Adds a title or description to the table. It appears above the table.
>	

- `<thead>` Groups the header content of a table (usually rows with `<th>`).
- `<tbody>` Groups the body content of the table (data rows with `<td>`).
- `<tfoot>` Groups the footer content of the table, often used for summaries or totals.
- `<colgroup>` Specifies groups of columns and applies styles or attributes to them.
`<col>` Defines the attributes for a single column within a `<colgroup>`.

Example:

A basic HTML table

Company	Contact	Country
Samera Fabric	Aryan Ashraf	Pakistan
Omega Shoes	Aryan Ashraf	UAE

```
<!DOCTYPE html>

<html>

<head>

<title> Basic Table </title>

</head>

<body>

<h2>A basic HTML table</h2>
```

```
<table style="width:100%">
  <tr>
    <th>Company</th>
    <th>Contact</th>
    <th>Country</th>
  </tr>
  <tr>
    <td> Samera Fabric</td>
    <td>Aryan Ashraf</td>
    <td>Pakistan</td>
  </tr>
  <tr>
    <td> Omega Shoes</td>
    <td> Aryan Ashraf </td>
    <td> UAE </td>
  </tr>
</table>
</body>
</html>
```

Explanation of Table:

1. **<table>**: Starts and ends the table.
2. **<caption>**: Adds a title ("Student Performance Report") above the table.

3. **<thead>**: Groups the header row with column names (Roll Number, Name, etc.).
4. **<tbody>**: Contains the main table data (student details and grades).
5. **<tfoot>**: Adds a footer row for totals or summary.
6. **border="1"**: Adds a border to the table for better visibility.

Key Attributes for Tables:

- **border**: Defines the thickness of the table border.
- **cellspacing**: Specifies the space between table cells.
- **cellpadding**: Defines the padding inside each cell.
- **width** and **height**: Set the size of the table.

How To Add a Border:

To add a border, use the CSS `border` property on `table`, `th`, and `td` elements:

Code :

```
<head>

<style>

table, th, td {

    border: 1px solid black;

}

</style>

</head>
```

Collapsed Table Borders:

To avoid having double borders like in the example above, set the CSS `border-collapse` property to `collapse`.

Code:

```
<head>
<style>
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
}
</style>
</head>
```

HTML Table Sizes

1. Set the width of the table to 100%:

100% wide HTML Table

Firstname	Lastname	Age
Aryan	Ashraf	22

```
<!DOCTYPE html>
<html>
<style>
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
}
</style>

<body>
```

<h2>100% wide HTML Table</h2>

```
<table style="width:100%">
```

```
<tr>
```

```
<th>Firstname</th>
```

```
<th>Lastname</th>
```

```
<th>Age</th>
```

```
</tr>
```

```
<tr>
```

```
<td> Aryan </td>
```

```
<td> Ashraf </td>
```

```
<td> 22 </td>
```

```
</tr>
```

```
</table>
```

```
</body>
```

```
</html>
```

2. Set the width of the first column to 70%:

Set the first column to 70% of the table width

Firstname	Lastname	Age
Aryan	Ashraf	22

```
<!DOCTYPE html>
```

```
<html>
```

```
<style>
```

```
table, th, td {
```

```
border:1px solid black;
```

```
border-collapse: collapse;
```

```
}
```

```
</style>
```

```
<body>
```

```
<h2>Set the first column to 70% of the table width</h2>
```

```
<table style="width:100%">
  <tr>
    <th style="width:70%">Firstname</th>
    <th>Lastname</th>
    <th>Age</th>
  </tr>
  <tr>
    <td> Aryan </td>
    <td> Ashraf </td>
    <td> 22 </td>
  </tr>
</table>

</body>
</html>
```

3. HTML Table Row Height:

Set the height of the second row to 200 pixels

Firstname	Lastname	Age
Aryan	Ashraf	22
Asim	Ali	22
Umair	Rehman	22

```
<!DOCTYPE html>
```

```
<html>
```

```
<style>
```

```
table, th, td {
```

```
  border:1px solid black;
```

```
  border-collapse: collapse;
```

```
}
```

```
</style>
```

```
<body>
```

```
<h2>Set the height of the second row to 200 pixels</h2>
```

```
<table style="width:100%">
```

```
<tr>
```

```
  <th>Firstname</th>
```

```
  <th>Lastname</th>
```

```
  <th>Age</th>
```

```
</tr>
```

```
<tr style="height:200px">
```

```
  <td> Aryan </td>
```

```
  <td> Ashraf </td>
```

```
  <td> 22 </td>
```

```
</tr>
```

```
<tr>
```

```
  <td> Asim </td>
```

```
  <td> Ali </td>
```

```
  <td> 22 </td>
```

```
</tr>
```

```
<tr>
```

```
  <td> Umair </td>
```

```
  <td> Rehman </td>
```

```
  <td> 22 </td>
```

```
</tr>
</table>

</body>
</html>
```

HTML Table - Cell Padding

Cellpadding

Cell padding specifies the space between the cell content and its borders.

Firstname	Lastname	Age
Aryan	Ashraf	22

```
<!DOCTYPE html>
<html>
<head>
<style>
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
}
th, td {
  padding: 20px;
}
</style>
</head>
<body>

<h2>Cellpadding</h2>
<p>Cell padding specifies the space between the cell content and its borders.</p>

<table style="width:100%">
  <tr>
    <th>Firstname</th>
    <th>Lastname</th>
    <th>Age</th>
  </tr>
  <tr>
    <td> Aryan </td>
```

```
<td> Ashraf </td>
<td> 22 </td>
</tr>
```

```
</table>
```

```
</body>
```

```
</html>
```

HTML Table - Cell Spacing

Cellspacing

Change the space between the cells with the border-spacing property.

Firstname	Lastname	Age
Aryan	Ashraf	22
Ali	Arshad	21

```
<!DOCTYPE html>
<html>
<head>
<style>
table, th, td {
  border: 1px solid black;
}
table {
  border-spacing: 30px;
}
</style>
</head>
<body>
```

```
<h2>Cellspacing</h2>
```

```
<p>Change the space between the cells with the border-spacing
property.</p>
```

```
<table style="width:100%">
```

```
<tr>
```

```
<th>Firstname</th>
```

```
<th>Lastname</th>
```

```
<th>Age</th>
```

```
</tr>
```

```
<tr>
```

```
<td> Aryan </td>
```

```
<td> Ashraf </td>
```

```
<td> 22 </td>
```

```
</tr>
```

```
<tr>
```

```
<td> Ali </td>
```

```
<td> Arshad </td>
```

```
<td> 21 </td>
```

```
</tr>
```

```
</table>
```

```
</body>
```

```
</html>
```

Colspan and Rowspan

Colspan:

Cell that spans two columns

To make a cell span more than one column, use the colspan attribute.

Name		Age
Aryan	Ashraf	22

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
table, th, td {
```

```
border: 1px solid black;
```

```
border-collapse: collapse;
```



```
}  
</style>  
</head>  
<body>
```

<h2>Cell that spans two columns</h2>

<p>To make a cell span more than one column, use the colspan attribute.</p>

```
<table style="width:100%">  
  <tr>  
    <th colspan="2">Name</th>  
    <th>Age</th>  
  </tr>  
  <tr>  
    <td> Aryan </td>  
    <td> Ashraf </td>  
    <td> 22 </td>  
  </tr>  
</table>  
</body>  
</html>
```

Rowspan:

Cell that spans two rows

To make a cell span more than one row, use the rowspan attribute.

Name	Aryan
Phone	03114030609
	03494106638

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
table, th, td {  
  border: 1px solid black;  
  border-collapse: collapse;  
}  
</style>  
</head>
```

```
<body>
```

```
<h2>Cell that spans two rows</h2>
```

```
<p>To make a cell span more than one row, use the rowspan attribute.</p>
```

```
<table style="width:100%">
```

```
<tr>
```

```
<th>Name</th>
```

```
<td> Aryan </td>
```

```
</tr>
```

```
<tr>
```

```
<th rowspan="2">Phone</th>
```

```
<td>03114030609</td>
```

```
</tr>
```

```
<tr>
```

```
<td>03494106638</td>
```

```
</tr>
```

```
</table>
```

```
</body>
```

```
</html>
```

Meta Tags

Meta tags are elements in an HTML document that provide metadata (information about the webpage) to search engines and browsers. These tags are placed inside the `<head>` section of the HTML document and do not appear on the webpage itself. Meta tags improve SEO (Search Engine Optimization) and help browsers understand the page content.

Example:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<meta name="description" content="A detailed tutorial on HTML and CSS  
for beginners.">
```

```
<meta name="keywords" content="HTML, CSS, Web Development,  
Frontend">
```

```
<meta name="author" content="John Doe">
```

```
<title>HTML Meta Tag Example</title>
```

```
</head>
```

```
<body>
```

```
<h1>Welcome to HTML Tutorial</h1>
```

```
<p>Learn the basics of HTML and CSS to create amazing websites!</p>
```

```
</body>
```

```
</html>
```

Explanation

- **<meta charset="UTF-8">**: Sets the character encoding of the webpage to support special characters.
- **<meta name="description">**: Provides a brief description of the page, often displayed in search engine results.
- **<meta name="keywords">**: Specifies relevant keywords for search engines to understand the page content.
- **<meta name="author">**: Indicates the author of the webpage.
- **<meta name="viewport">**: Ensures the page is responsive and adjusts properly to different screen sizes.

Form Tags

Form tags are used to create a form on a webpage, enabling user input collection. Forms can include text fields, checkboxes, buttons, and other input elements. The data collected is typically sent to a server for processing.

Example; Sign Up Page:

Contact Information

First Name:

Last Name:

Select Gender: ☐ Female ☐ Male

Employee ID:

Designation:

Phone No:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My Profile Page</title>
```

```
</head>
```

```
<body><center>
```

```
<h1 >Contact Information</h1>
```

```
<Form action="loginpage.html" method="post">
```

```
<Label for "fname">First Name:</label>
```

```
<input type= "text" id="fname" name="fname" required><br><br>
```

```
<Label for "lname">Last Name:</label>
```

```
<input type= "text" id="lname" name="lname"required><br></br>
```

```
<Label for "select gender">Select Gender:</label>
```

```
<input type="radio" id="female" name="gender" value="Female"  
required>
```

```
<label for="female">Female</label>
```

```
<input type="radio" id="male" name="gender" value="Male" required>
```

```
<label for="male" >Male</label><br><br>
```

```
<Label for "empid">Employee ID:</label>
```

```
<input type= "text" id="empid" name="empid" required><br><br>
```

```
<Label for "Designation">Designation:</label>
```

```
<input type= "text" id="Designation" name="Designation"  
required><br><br>
```

```
<Label for "Phone no">Phone No:</label>
```



```
<input type= "text" id="Phone no" name="Phone no"
required><br><br>
```

```
<input type ="submit"> </center>

</form>

</body>

</html>
```

Explanation

- **<form>**: Defines the start and end of the form.
- **action**: Specifies the URL where the form data is sent.
- **method**: Defines the HTTP method (e.g., **post**, **get**) for sending data.
- **<label>**: Describes the purpose of the input fields.
- **<input>**: Allows users to input text, emails, passwords, or other data.
- **<textarea>**: Provides a multi-line input field for longer text.
- **<button>**: Creates buttons for submitting or resetting the form.

Common Input Types:

<input type="text">: Accepts single-line text input.

<input type="password">: Masks text for password entry.

<input type="email">: Accepts email addresses.

<input type="checkbox">: Creates checkboxes for multiple selections.

<input type="radio">: Allows selecting one option from a group.

<input type="submit">: Sends the form data to the server.

<input type="reset">: Resets the form fields to their initial values.

<input type="button">: Creates a customizable button.

Sign In

Contact Information

First Name:

Phone No:

Submit

```
<!DOCTYPE html>
<html>

<head>
  <title>My Profile Page</title>
</head>

<body> <center>
<h1>Contact Information</h1>
  <Form action="Practice1.html" method="post">

    <Label for "fname">First Name:</label>
    <input type= "text" id="fname" name="fname" required><br><br>

    <Label for "Phone no">Phone No:</label>
    <input type= "text" id="Phone no" name="Phone no"
required><br><br>

    <input type ="submit"></center>

  </form>

</body>
```

</html>

Common Elements:

1. **<input>**: Collects user input in various formats.
2. **<label>**: Provides labels for form fields.
3. **<select>**: Creates dropdown menus.
4. **<textarea>**: Allows multi-line text input.
5. **<button>**: Adds clickable buttons.
6. **<option>**: Specifies options in a dropdown menu.

Input:

The input Element

First name:

Submit

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>The input Element</h2>
```

```
<label for="fname">First name:</label><br>
```

```
<input type="text" id="fname" name="fname"><br><br>
```

```
<input type="submit" value="Submit">
```

```
</form>
```

```
</body>
```

```
</html>
```

Select:

What Is Your Department

Choose a Department:

- BBA
- Commerce
- English
- IT

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2> What Is Your Department </h2>
```

```
<label for="Department">Choose a Department:</label>
```

```
<select id="Department" name=" Department">
```

```
<option value="BBA">BBA</option>
```

```
<option value="Commerce">Commerce</option>
```

```
<option value="English">English</option>
```

```
<option value=" IT">IT</option>
```

```
</select>
```

```
<input type="submit">
```

```
</form>
```

```
</body>
```

```
</html>
```

Textarea:

Textarea

Ladies and Gentleman I
would like to introduce my
self.

Submit

```
<!DOCTYPE html>
<html>
<body>

<h2>Textarea</h2>

  <textarea name="message" rows="8" cols="25">Ladies and Gentleman I would like
to introduce my self.</textarea>
  <br><br>
  <input type="submit">
</form>

</body>
</html>
```

Button Element:

The button Element

Press!

```
<!DOCTYPE html>

<html>

<body>
```



```
<h2>The button Element</h2>
```

```
<button type="button" onclick="alert('Good Morning!')">Press!</button>
```

```
</body>
```

```
</html>
```

Links In HTML

HyperLink:

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h1> Hyper Link </h1>
```

```
<a href="https://www.yourwebsitelinkurl/">visit your website!</a><br><br>
```

```
</body>
```

```
</html>
```

EmailLink:

```
<!DOCTYPE html>

<html>

<body>

<h1> Email Link </h1>

<a href="mailto:someone@example.com">Send email</a>

</body>

</html>
```

ImageLink:

```
<!DOCTYPE html>

<html>

<body>

<h1> Email Link </h1>



</body>

</html>
```

CSS (Cascading Style Sheets)

CSS, or Cascading Style Sheets, is a language used to style and design the appearance of HTML elements on a webpage. It controls the layout, colors, fonts, and overall look of a website. CSS enhances the visual appeal of web pages and ensures they are consistent and attractive.

Functions of CSS

1. **Styling Text:** Customize font size, style, color, and alignment of text.
2. **Changing Backgrounds:** Add background colors, images, or gradients.
3. **Adjusting Layouts:** Position elements using grids, flexbox, or other layout techniques.
4. **Enhancing Responsiveness:** Make web pages adaptable to various devices like mobiles, tablets, and desktops.
5. **Adding Animations:** Create smooth transitions, animations, and hover effects.
6. **Improving Usability:** Style buttons, forms, and links to make the website user-friendly.

Methods of Applying CSS

CSS can be applied in three main ways: **Inline CSS**, **Internal CSS**, and **External CSS**. Each has its unique use cases.

1. Inline CSS

Inline CSS is written directly inside the HTML element using the **style** attribute. It is typically used for quick or small styling needs.

Example:

```
<!DOCTYPE html>
<html>
<body>

<h1 style="color:blue;">A Blue Heading</h1>

<p style="color:red;">A red paragraph.</p>

</body>
</html>
```

When to Use:

- For minor or temporary changes.
- When only a specific element needs to be styled.

Drawback:

- Hard to maintain if applied to many elements.
-

2. Internal CSS

In internal CSS, CSS rules are written within a `<style>` tag in the `<head>` section of the HTML document. It is useful for styling a single webpage.

Example:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {background-color:offwhite;}
h1   {color: red;}
p    {color:green;}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

When to Use:

- For small projects or single-page websites.

Drawback:

- Styles cannot be reused across multiple pages.

3. External CSS

In external CSS, CSS rules are written in a separate `.css` file and linked to the HTML document using the `<link>` tag. This method ensures all pages of a website have a consistent design.

Example:

```
<!DOCTYPE html>

<html>

<head>

  <link rel="stylesheet" href="styles.css">

</head>

<body>

<h1>This is a heading</h1>

<p>This is a paragraph.</p>

</body>

</html>
```

When to Use:

- For large websites with multiple pages.
- When maintaining a consistent design across all pages.

Advantages:

- Makes code cleaner and easier to maintain.
- Allows reusability of CSS across different pages.

Drawback:

- Requires an additional HTTP request to load the CSS file, slightly
- affecting load times if not optimized.

Class Paractice CSS

Inline Css:

Contact Information

First Name:	
Last Name:	
Select Gender:	☐ Female ☐ Male
Employee ID:	
Designation:	
Phone No:	
<input type="submit" value="Submit"/>	

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>My Profile Page</title>
```

```
<style>
```

```
body {
```

```
    font-family: Arial, sans-serif;
```

```
    margin: 20px;
```

```
}
```

```
h1 {
```

```
    color: blue;
```

```
}
```

```
form {
```

```
    background-color: #f9f9f9;
```

```
    padding: 20px;
```

```
    border: 1px solid #ccc;
```

```
    width: 300px;
```

```
    text-align: left;
```

```
    margin: auto;
```

```
}
```

```
label {
```

```
    display: block;
```

```
    margin-bottom: 5px;
```

```
}
```

```
input[type="text"] {  
    width: 100%;  
    padding: 5px;  
    margin-bottom: 15px;  
}
```

```
input[type="submit"] {  
    background-color: blue;  
    color: white;  
    padding: 10px 15px;  
    border: none;  
    cursor: pointer;  
}
```

```
input[type="radio"] {  
    margin-right: 5px;  
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<center>
```

```
<h1>Contact Information</h1>
```

```
<form action="loginpage.html" method="post">
```

```
<label for="fname">First Name:</label>
```

<input type="text" id="fname" name="fname" required>

<label for="lname">Last Name:</label>

<input type="text" id="lname" name="lname" required>

<label>Select Gender:</label>

<input type="radio" id="female" name="gender" value="Female" required>

<label for="female" style="display: inline;">Female</label>

<input type="radio" id="male" name="gender" value="Male" required>

<label for="male" style="display: inline;">Male</label>

<label for="empid">Employee ID:</label>

<input type="text" id="empid" name="empid" required>

<label for="Designation">Designation:</label>

<input type="text" id="Designation" name="Designation" required>

<label for="Phone no">Phone No:</label>

<input type="text" id="Phone no" name="Phone no" required>

<input type="submit">

</form>

</center>

</body>

</html>

Internal CSS:

Contact Information

First Name:	
Last Name:	
Select Gender:	☐ Female ☐ Male
Employee ID:	
Designation:	
Phone No:	
<input type="submit" value="Submit"/>	

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My Profile Page</title>
```

```
</head>
```

```
<body style="font-family: Arial, sans-serif; margin: 20px;">
```

```
  <center>
```

```
    <h1 style="color: blue;">Contact Information</h1>
```

```
    <!-- this is my form page-->
```



```
<form action="loginpage.html" method="post" style="background-color: #f9f9f9;
padding: 20px; border: 1px solid #ccc; width: 300px; text-align: left;">
  <!-- this is my form -->
    <label for="fname" style="display: block; margin-bottom: 5px;">First
Name:</label>
    <input type="text" id="fname" name="fname" required style="width: 100%;
padding: 5px; margin-bottom: 15px;"><br>

    <label for="lname" style="display: block; margin-bottom: 5px;">Last
Name:</label>
    <input type="text" id="lname" name="lname" required style="width: 100%;
padding: 5px; margin-bottom: 15px;"><br>

    <label style="display: block; margin-bottom: 5px;">Select Gender:</label>
    <input type="radio" id="female" name="gender" value="Female" required>
    <label for="female" style="margin-right: 10px;">Female</label>
    <input type="radio" id="male" name="gender" value="Male" required>
    <label for="male">Male</label><br><br>

    <label for="empid" style="display: block; margin-bottom: 5px;">Employee
ID:</label>
    <input type="text" id="empid" name="empid" required style="width: 100%;
padding: 5px; margin-bottom: 15px;"><br>

    <label for="Designation" style="display: block; margin-bottom:
5px;">Designation:</label>
    <input type="text" id="Designation" name="Designation" required
style="width: 100%; padding: 5px; margin-bottom: 15px;"><br>

    <label for="Phone no" style="display: block; margin-bottom: 5px;">Phone
No:</label>
    <input type="text" id="Phone no" name="Phone no" required style="width:
100%; padding: 5px; margin-bottom: 15px;"><br>

    <input type="submit" style="background-color: blue; color: white; padding:
10px 15px; border: none; cursor: pointer;">
  </form>
</center>
</body>
</html>
```

Website:

Website is a collection of related web pages that may contain text, images, audio and video. The first page of a website is called home page. Each website has specific internet address (URL) that you need to enter in your browser to access a website.

Website is hosted on one or more servers and can be accessed by visiting its homepage using a computer network. A website is managed by its owner that can be an individual, company or an organization.

A website can be of two types:

- Static Website
- Dynamic Website

Static website:

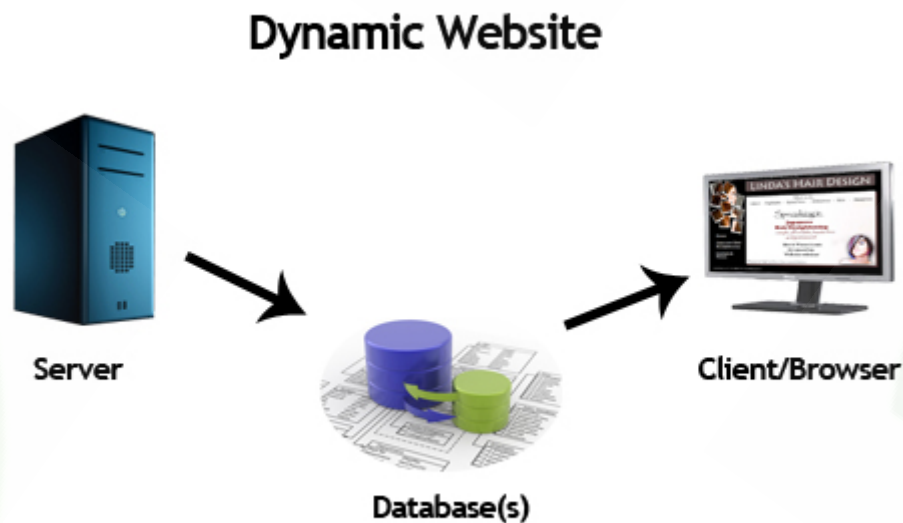
Static website is the basic type of website that is easy to create. You don't need the knowledge of web programming and database design to create a static website. Its web pages are coded in HTML. The codes are fixed for each page so the information contained in the page does not change and it looks like a printed page.



Dynamic website:

Dynamic website is a collection of dynamic web pages whose content changes dynamically. It accesses content from a database or Content Management System (CMS). Therefore, when you alter or update the content of the database, the content of the website is also altered or updated.

Dynamic website uses client-side scripting or server-side scripting, or both to generate dynamic content.



Static vs Dynamic website:

Static Website

Prebuilt content is same every time the page is loaded.

It uses the HTML code for developing a website.

It sends exactly the same response for every request.

Dynamic Website

Content is generated quickly and changes regularly.

It uses the server side languages such as PHP, SERVLET, JSP, and ASP.NET etc. for developing a website.

It may generate different HTML for each of the request.

The content is only changed when someone publishes and updates the file (sends it to the web server).

The page contains "server-side" code which allows the server to generate the unique content when the page is loaded.

Flexibility is the main advantage of static website.

Content Management System (CMS) is the main advantage of dynamic website.

Servlet:

A servlet is a Java program that runs on a server to handle requests and generate responses, typically for web applications. Servlets are part of the Java EE platform and are used to create dynamic web content. They process user inputs, interact with databases, and generate outputs like HTML, JSON, or XML to send back to the client.

Example:

A servlet can be used to authenticate users by verifying their credentials against a database.

JSP (JavaServer Pages):

JSP is a server-side technology for building dynamic web pages. It allows embedding Java code directly into HTML pages, making it easier to develop user interfaces. A JSP file is compiled into a servlet before execution, so it works seamlessly with other Java technologies.

Example:

A JSP page can be used to display user profile information fetched from a database.

Differences Between Servlet and JSP

Aspect	Servlet	JSP
Definition	Java programs used to handle client-server interactions.	Web pages that combine Java and HTML to generate dynamic content.
Code Structure	Contains more Java code and is less HTML-friendly.	Primarily an HTML page with embedded Java code, easier for designers.
Ease of Development	More complex to write and maintain due to heavy Java coding.	Easier to write because it focuses on HTML with Java snippets.
Use Case	Used for processing logic, such as handling requests, validating data, or interacting with databases.	Used for presentation logic, such as displaying processed data in a user-friendly format.
Execution	Directly compiled and executed by the server.	First converted into a servlet and then executed.